



Engenharia de Computação
Fundamentos de Processamento Digital de Imagens

**Deteccção de Objetos com Redes Profundas: Estudo Comparativo
entre YOLOv8n e SSDLite320 MobileNetV3 Large**

Caio Vinicius – 558169
Guilherme Beserra – 538660
Marina Vasques – 496500
Jennifer Gadelha – 536294

Fortaleza
2026

Sumário

1	INTRODUÇÃO	3
2	OBJETIVOS	4
2.1	Objetivo Geral	4
2.2	Objetivos Específicos	4
3	FUNDAMENTAÇÃO TEÓRICA	5
3.1	Fundamentos da Imagem Digital	5
3.2	Detecção de Objetos com Redes Profundas	5
3.3	YOLOv8n	6
3.4	SSDLite320 MobileNetV3 Large	6
3.5	Espaços de Cores e Pré-processamento	7
3.6	Data Augmentation	7
3.7	Segmentação e Morfologia Matemática	7
3.8	Métricas de Avaliação	8
4	METODOLOGIA	10
4.1	Bases de Dados	10
4.2	Arquiteturas Utilizadas	10
4.3	Ambiente Computacional	10
4.4	Pipeline de Pré-processamento Padronizado	11
4.5	Hiperparâmetros	11
4.6	Critério de Early Stopping	12
4.7	Threshold de Confiança e Cálculo de mAP	12
4.8	Estrutura dos Experimentos	12
4.9	Demonstração nas Três Modalidades	13
5	RESULTADOS E DISCUSSÃO	15
5.1	Métricas — YOLOv8n	15
5.2	Métricas — SSDLite320 MobileNetV3 Large	15
5.3	Comparação Final entre as Redes	16
5.4	Análise dos Resultados	16
5.4.1	Impacto do Data Augmentation	16
5.4.2	Impacto do Pré-processamento	17
5.4.3	Impacto dos Hiperparâmetros Modificados	17
5.4.4	Comparação entre Arquiteturas	18

5.4.5	Dificuldade da Base de Canetas	18
5.4.6	Tempo de Inferência	19
5.4.7	Qualidade da Segmentação Pós-Detecção	19
6	CONCLUSÃO	20
	REFERÊNCIAS	21
	APÊNDICES	22
	APÊNDICE A – RESULTADOS VISUAIS DAS DETECÇÕES . . .	23
A.1	Resultados Visuais dos Experimentos	23
A.1.1	YOLOv8n — Base de Canetas	23
A.1.2	YOLOv8n — Base de Maças	23
A.1.3	SSDLite320 — Base de Canetas	24
A.1.4	SSDLite320 — Base de Maças	24
A.2	Testes da Aplicação Web	25
A.2.1	YOLOv8n — Testes na Base de Maças	25
A.2.2	SSDLite320 — Testes na Base de Canetas	25
	APÊNDICE B – ESTRUTURA DOS NOTEBOOKS COLAB	26
	APÊNDICE C – REPOSITÓRIO E ORGANIZAÇÃO DOS ARQUI- VOS	27

1 Introdução

A detecção de objetos é uma das tarefas fundamentais da visão computacional, consistindo em identificar e localizar instâncias de objetos em imagens ou vídeos por meio de *bounding boxes* (caixas delimitadoras). Com o avanço das redes neurais convolucionais profundas, surgiram arquiteturas altamente eficientes para essa tarefa, com diferentes compromissos entre acurácia e velocidade de inferência.

Este trabalho apresenta um estudo comparativo entre duas arquiteturas de redes profundas para detecção de objetos, YOLOv8n e SSDLite320 MobileNetV3 Large, aplicadas a duas bases de dados distintas: canetas e maçãs. O experimento baseia-se em um *pipeline* metodológico composto por seis configurações experimentais, nas quais variam-se parâmetros de pré-processamento, hiperparâmetros de treinamento e técnicas de data augmentation, a fim de avaliar o impacto de cada componente sobre o desempenho final de cada arquitetura.

O desenvolvimento foi conduzido em paralelo por dois grupos: a Dupla A ficou responsável pela rede YOLOv8n, enquanto a Dupla B foi encarregada da SSDLite320 MobileNetV3 Large. Ambos os grupos empregaram as mesmas bases de dados, o mesmo protocolo de avaliação, 5 execuções independentes, com cálculo de média e desvio padrão, e o mesmo pipeline de pré-processamento, o que assegura a comparabilidade dos resultados obtidos.

Além dos experimentos conduzidos em ambiente de desenvolvimento, Google Colab com GPU T4, o trabalho contempla a demonstração em três modalidades de entrada, imagem estática, vídeo e *webcam* ao vivo, implementadas por meio de uma aplicação web com backend em *FastAPI* e *frontend* em *React/TypeScript*, cujo código-fonte está disponível no repositório público do grupo¹.

¹ Repositório disponível em: <https://github.com/guilhermebsantiago/pdi-deteccao-objetos>

2 Objetivos

2.1 Objetivo Geral

Comparar o desempenho das arquiteturas YOLOv8n e SSDLite320 MobileNetV3 Large na tarefa de detecção de objetos em duas bases de dados distintas, avaliando o impacto de pré-processamento, variação de hiperparâmetros e *data augmentation* manual.

2.2 Objetivos Específicos

- Implementar e treinar as redes YOLOv8n e SSDLite320 MobileNetV3 Large nas bases de caneta e maçã;
- Avaliar seis configurações experimentais para cada rede e base, reportando Precision, Recall, F1-Score, mAP@0.50 e mAP@0.50:0.95 com média e desvio padrão de 5 execuções independentes;
- Implementar manualmente técnicas de *data augmentation* (*flip* horizontal, ajuste de brilho, ruído gaussiano e rotação), sem uso de bibliotecas prontas como *albumentations* ou *imgaug*;
- Aplicar pós-processamento com segmentação sobre as detecções, utilizando técnicas clássicas de PDI (CLAHE, binarização Otsu, detecção de bordas Canny e morfologia matemática);
- Demonstrar o sistema treinado nas três modalidades exigidas: imagem estática, vídeo e *webcam* ao vivo.

3 Fundamentação Teórica

3.1 Fundamentos da Imagem Digital

Uma imagem digital pode ser definida como uma função bidimensional $f(x, y)$, em que x e y representam coordenadas espaciais e a amplitude da função em cada ponto corresponde à intensidade luminosa ou nível de cinza da imagem [Gonzalez e Woods 2018]. Quando os valores de x , y e da intensidade assumem valores discretos e finitos, a imagem é denominada imagem digital.

O processo de conversão de uma imagem contínua em uma imagem digital envolve duas etapas fundamentais: amostragem e quantização. A amostragem consiste na discretização das coordenadas espaciais da imagem, enquanto a quantização discretiza os valores de intensidade. O resultado é uma matriz de elementos pictóricos, denominados pixels, que constituem a menor unidade representativa da imagem digital.

No contexto do processamento digital de imagens, diferentes operações podem ser aplicadas sobre os pixels com o objetivo de melhorar a qualidade visual, reduzir ruídos ou destacar regiões de interesse. Essas operações incluem transformações espaciais, filtragem, segmentação e análise morfológica, amplamente utilizadas em sistemas de visão computacional e detecção de objetos.

3.2 Detecção de Objetos com Redes Profundas

A detecção de objetos é uma tarefa da visão computacional que consiste em localizar e classificar objetos presentes em imagens ou vídeos. Diferentemente da classificação tradicional, que atribui apenas uma classe à imagem inteira, a detecção realiza simultaneamente a identificação do objeto e a estimativa de sua posição por meio de bounding boxes.

As arquiteturas modernas de detecção são geralmente divididas em métodos de dois estágios e métodos de um estágio. Nos métodos de dois estágios, como Faster R-CNN, a rede inicialmente propõe regiões candidatas e, posteriormente, realiza a classificação dessas regiões. Embora apresentem elevada precisão, tais métodos possuem maior custo computacional.

Por outro lado, métodos de um estágio, como YOLO e SSD, realizam localização e classificação em uma única passagem pela rede neural, reduzindo significativamente o tempo de inferência e tornando-se mais adequados para aplicações em tempo real [Redmon et al. 2016, Liu et al. 2016].

As redes convolucionais profundas (CNNs) são a base dessas arquiteturas. Nessas redes, convoluções sucessivas permitem extrair características hierárquicas da imagem, como

bordas, texturas, formas e padrões complexos. O treinamento ocorre por aprendizado supervisionado, utilizando imagens anotadas com classes e coordenadas das bounding boxes.

3.3 YOLOv8n

A família YOLO (You Only Look Once) é uma das arquiteturas mais difundidas para detecção em tempo real [Redmon et al. 2016]. A principal característica dessa abordagem é tratar a detecção como um único problema de regressão, no qual a rede prediz simultaneamente classes e coordenadas das caixas delimitadoras.

A variante YOLOv8n (nano) representa a menor e mais leve versão da oitava geração da arquitetura YOLO, desenvolvida pela Ultralytics [Jocher, Chaurasia e Qiu 2023]. Sua estrutura utiliza uma backbone baseada em CSPNet (Cross Stage Partial Network), cuja proposta é melhorar o fluxo de gradientes e reduzir redundâncias computacionais, aumentando a eficiência da rede.

Além disso, a YOLOv8 emprega uma abordagem *anchor-free*, eliminando a necessidade de caixas âncora pré-definidas. Essa estratégia reduz a complexidade do processo de treinamento e melhora a adaptação a diferentes tamanhos de objetos.

A arquitetura também utiliza estruturas do tipo FPN (Feature Pyramid Network) e PAN (Path Aggregation Network), responsáveis por combinar informações de múltiplas escalas da imagem, permitindo melhor detecção tanto de objetos pequenos quanto grandes.

3.4 SSDLite320 MobileNetV3 Large

A arquitetura SSD (Single Shot MultiBox Detector) realiza a detecção de objetos em múltiplas escalas utilizando mapas de características extraídos de diferentes profundidades da rede [Liu et al. 2016]. Dessa forma, objetos de diferentes tamanhos podem ser detectados simultaneamente.

A variante SSDLite320 MobileNetV3 Large combina o detector SSD com a backbone MobileNetV3 Large, projetada especificamente para dispositivos móveis e sistemas embarcados [Howard et al. 2019]. Essa arquitetura utiliza convoluções *depthwise* separáveis, nas quais a convolução espacial e a convolução entre canais são realizadas separadamente.

Em convoluções tradicionais, o custo computacional cresce proporcionalmente ao número de canais de entrada e saída. Já nas convoluções *depthwise* separáveis, esse custo é drasticamente reduzido, diminuindo o número de parâmetros e operações matemáticas necessárias. Como consequência, a MobileNetV3 apresenta menor consumo de memória e maior velocidade de inferência.

A entrada da SSDLite320 é fixada em 320×320 pixels, requisito arquitetural utilizado para padronizar o tamanho das imagens e garantir eficiência computacional.

3.5 Espaços de Cores e Pré-processamento

Uma imagem colorida pode ser representada em diferentes espaços de cores. O modelo RGB representa cada pixel a partir das componentes vermelho (Red), verde (Green) e azul (Blue). Embora amplamente utilizado para aquisição e exibição de imagens, o RGB não separa explicitamente a informação de intensidade luminosa da informação cromática.

Em aplicações de PDI, modelos como HSI e LAB são frequentemente preferidos, pois desacoplam brilho e cor. No espaço LAB, por exemplo, o canal L representa a luminância, enquanto os canais A e B armazenam informações cromáticas.

Essa separação permite aplicar transformações de intensidade sem alterar significativamente as cores originais da imagem. Neste trabalho, foi utilizada a técnica CLAHE (Contrast Limited Adaptive Histogram Equalization) sobre o canal L do espaço LAB.

O CLAHE realiza equalização adaptativa de histograma em pequenas regiões da imagem, aumentando o contraste local sem amplificar excessivamente o ruído [Gonzalez e Woods 2018]. Diferentemente da equalização global tradicional, o método limita o ganho máximo de contraste por meio do parâmetro clip limit, evitando saturação excessiva.

Após o CLAHE, foi aplicado blur gaussiano, filtro linear baseado na convolução da imagem com uma função gaussiana. Esse filtro reduz componentes de alta frequência associadas a ruídos, suavizando a imagem antes do processamento posterior.

3.6 Data Augmentation

Data augmentation consiste na criação artificial de novas amostras de treinamento a partir de transformações aplicadas às imagens originais. Essa técnica aumenta a variabilidade dos dados disponíveis, reduzindo problemas de overfitting e melhorando a capacidade de generalização das redes neurais.

As transformações utilizadas incluem flip horizontal, ajuste de brilho, adição de ruído gaussiano e rotação. O flip horizontal realiza o espelhamento da imagem e exige o recálculo das coordenadas das bounding boxes. O ajuste de brilho altera os níveis de intensidade luminosa dos pixels, simulando diferentes condições de iluminação.

O ruído gaussiano modela imperfeições presentes em sensores e sistemas de captura. Já a rotação foi aplicada em ângulos aleatórios entre -20° e $+20^\circ$, exigindo o recálculo geométrico das bounding boxes a partir dos quatro vértices originais.

3.7 Segmentação e Morfologia Matemática

A segmentação é o processo de particionar uma imagem em regiões significativas, permitindo separar objetos do fundo. Neste projeto, a segmentação foi aplicada após a detecção dos objetos, utilizando um pipeline clássico de processamento digital de imagens.

A binarização por Otsu é uma técnica automática de limiarização que seleciona o limiar ótimo a partir do histograma da imagem [Gonzalez e Woods 2018]. O método busca maximizar a variância entre classes (σ_B^2), separando estatisticamente pixels de fundo e objeto com mínima sobreposição.

Após a binarização, foi utilizado o detector de bordas de Canny. Esse método baseia-se em três objetivos principais: baixa taxa de erro, boa localização das bordas e resposta única por borda detectada. O algoritmo envolve suavização gaussiana, cálculo do gradiente, supressão de não máximos e limiarização por histerese.

Na sequência, foram aplicadas operações de morfologia matemática. A morfologia utiliza conceitos da teoria dos conjuntos para analisar estruturas geométricas presentes na imagem. A erosão reduz regiões binárias, removendo pequenos ruídos e contraindo objetos. Já a operação de fechamento combina dilatação seguida de erosão, preenchendo pequenos buracos e conectando descontinuidades estreitas.

Por fim, os contornos foram extraídos e desenhados sobre os objetos detectados, produzindo uma segmentação visual da região de interesse.

3.8 Métricas de Avaliação

A avaliação de detectores de objetos utiliza métricas derivadas da matriz de confusão. A precisão (*Precision*) mede a proporção de detecções corretas dentre todas as detecções realizadas:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3.1)$$

em que TP representa verdadeiros positivos e FP falsos positivos.

O recall mede a proporção de objetos reais corretamente detectados:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3.2)$$

em que FN representa falsos negativos.

O F1-Score corresponde à média harmônica entre precisão e recall:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3.3)$$

A métrica IoU (*Intersection over Union*) mede a sobreposição entre a bounding box predita e a bounding box real:

$$\text{IoU} = \frac{\text{Área da Interseção}}{\text{Área da União}} \quad (3.4)$$

Um objeto é geralmente considerado corretamente detectado quando o IoU ultrapassa um limiar mínimo, tipicamente 0,50.

O mAP (mean Average Precision) corresponde à média das áreas sob as curvas Precisão-Recall para todas as classes avaliadas. A métrica mAP@0.50 utiliza limiar fixo de IoU igual a 0,50, enquanto o mAP@0.50:0.95 calcula a média em múltiplos limiares entre 0,50 e 0,95, sendo mais rigorosa e amplamente utilizada no benchmark COCO [Lin et al. 2014].

Além das métricas de acurácia, também foram avaliados tempo de treinamento, tempo de inferência e número de épocas executadas antes do *early stopping*, permitindo comparar eficiência computacional e desempenho preditivo das arquiteturas.

4 Metodologia

4.1 Bases de Dados

Base de Canetas (escolhida pela equipe): A base apresenta cenários variados e complexos, com canetas em diferentes posições e ângulos sobre fundos com outros objetos, tornando-a desafiadora para detecção.

Disponibilizada no Roboflow (*workspace*: packiot, projeto: pen-wwfeu, versão 1) [Roboflow]. Composição: 350 imagens de treino, 100 de validação e 50 de teste. Formato: YOLO txt (coordenadas relativas).

Base de Maças (fornecida pelo professor): A base apresenta maçãs em destaque, com fundo relativamente uniforme, sendo visualmente mais simples que a base de canetas.

Disponibilizada no Roboflow (*workspace*: marina-vasques-rodrigues, projeto: apple-c3cv9-pnvhy, versão 1) [Roboflow]. Composição: 97 imagens de treino, 27 de validação e 14 de teste. Formato: YOLO txt.

4.2 Arquiteturas Utilizadas

A Tabela 1 apresenta uma comparação dos principais parâmetros das arquiteturas empregadas.

Tabela 1 – Comparação das arquiteturas utilizadas

Parâmetro	YOLOv8n	SSDLite320 MobileNetV3
Framework	Ultralytics	PyTorch (torchvision)
Backbone	CSPNet	MobileNetV3 Large
Pré-treinamento	Sem pré-treinamento	ImageNet (backbone)
Tamanho de entrada	320×320	320×320
Classes	2 (bg + objeto)	2 (bg + objeto)

Fonte: Elaborada pelos autores.

4.3 Ambiente Computacional

Todos os experimentos foram executados no Google Colab com GPU NVIDIA T4 (gratuita). A mudança de CPU para GPU reduziu o tempo médio de treino de aproximadamente 4.000 segundos para 170 segundos por execução, viabilizando as 5 rodadas de cada experimento.

4.4 Pipeline de Pré-processamento Padronizado

O pré-processamento foi padronizado entre as duas redes e as duas bases, garantindo que diferenças de resultado sejam atribuídas às arquiteturas e não ao tratamento diferenciado dos dados. O *pipeline* aplicado é:

1. Redimensionamento para 320×320 *pixels* (interpolação bilinear);
2. Conversão para espaço de cores RGB;
3. CLAHE no canal L do espaço LAB ($\text{clipLimit} = 2,0$; $\text{tileGridSize} = 8 \times 8$);
4. *Blur* Gaussiano leve ($\text{kernel} 3 \times 3$; $\sigma = 0,8$).

Os arquivos de *labels* (.txt) foram copiados sem alteração, pois as coordenadas YOLO são relativas ao tamanho da imagem e invariantes ao redimensionamento.

4.5 Hiperparâmetros

Os hiperparâmetros foram padronizados entre as duas redes, respeitando as restrições arquiteturais de cada *framework*. O *batch size* da SSDLite é menor nos experimentos padrão por limitação de memória da GPU, com imagens 320×320 em PyTorch puro (Tabelas 2 e 3).

Tabela 2 – Hiperparâmetros padrão (Experimentos 1 e 3)

Parâmetro	YOLOv8n	SSDLite320
epochs	60	60
patience	12	12
<i>batch size</i>	16	8
imgsz	320	320
optimizer	AdamW	AdamW
lr	padrão Ultralytics	10^{-4}

Fonte: Elaborada pelos autores.

Tabela 3 – Hiperparâmetros modificados (Experimentos 2, 4, 5 e 6)

Parâmetro	YOLOv8n	SSDLite320
epochs	150	150
patience	20	20
<i>batch size</i>	16	16
imgsz	320	320
optimizer	AdamW	AdamW
lr	5×10^{-4}	5×10^{-4}

Fonte: Elaborada pelos autores.

4.6 Critério de Early Stopping

Ambas as redes utilizam *early stopping* com o mesmo critério proporcional: o treino é interrompido se a *loss* de validação não melhorar por **patience** épocas consecutivas, salvando sempre o melhor modelo encontrado. Na SSDLite, o *early stopping* foi implementado manualmente em PyTorch, replicando o comportamento nativo da Ultralytics na YOLOv8.

4.7 Threshold de Confiança e Cálculo de mAP

Na YOLOv8, o método **model.val()** da Ultralytics utiliza internamente um *threshold* baixo ($\approx 0,001$) para coletar todas as detecções e montar a curva precisão-*recall* usada no cálculo do mAP, e um *threshold* operacional mais alto para reportar Precision e Recall.

Na implementação da SSDLite foram utilizados dois limiares de confiança distintos. O parâmetro **CONF_THRESHOLD_EVAL = 0,01** foi empregado exclusivamente no cálculo do mAP, permitindo considerar praticamente todas as detecções geradas pela rede durante a construção da curva Precisão-Recall. Já o parâmetro **CONF_THRESHOLD = 0,3** foi utilizado nas métricas operacionais (Precision, Recall e F1-Score) e nas demonstrações visuais, filtrando detecções de baixa confiança e reduzindo a ocorrência de falsos positivos.

Essa separação garante que o mAP das duas redes seja calculado de forma conceitualmente equivalente, tornando a comparação justa.

4.8 Estrutura dos Experimentos

Foram realizados seis experimentos para cada rede e cada base de dados, totalizando 24 configurações experimentais. Cada experimento foi executado 5 vezes com *seeds* fixas e independentes, reportando média e desvio padrão (Tabela 4).

Tabela 4 – Estrutura dos experimentos

Exp.	Configuração
Exp. 1	Hiperparâmetros padrão, sem pré-processamento (linha de base)
Exp. 2	Hiperparâmetros modificados, sem pré-processamento
Exp. 3	Hiperparâmetros padrão, com pré-processamento (CLAHE + blur)
Exp. 4	Hiperparâmetros modificados, com pré-processamento
Exp. 5	Hiperparâmetros modificados, com <i>data augmentation</i> manual
Exp. 6	Hiperparâmetros modificados, com pré-processamento + <i>data augmentation</i> (configuração completa)

Fonte: Elaborada pelos autores.

O Exp. 1 serve como linha de base para medir o impacto isolado de cada componente nos experimentos seguintes. O isolamento de variáveis entre experimentos consecutivos permite atribuir ganhos ou perdas de desempenho a causas específicas.

4.9 Demonstração nas Três Modalidades

A etapa de demonstração foi implementada por meio de uma aplicação *web* desenvolvida para permitir a utilização prática dos modelos treinados em diferentes cenários de entrada. A arquitetura do sistema foi dividida em duas camadas principais:

- **Backend:** desenvolvido em Python utilizando o framework FastAPI, responsável pelo carregamento dos modelos treinados (`.pth` para a SSDLite320 MobileNetV3 Large e `.pt` para a YOLOv8n), processamento das imagens e execução da inferência;
- **Frontend:** desenvolvido em React com TypeScript, responsável pela interface gráfica e interação com o usuário nas três modalidades de demonstração.

A aplicação foi estruturada para operar em três modalidades distintas:

- **Modalidade Imagem:** o usuário realiza o envio de uma imagem estática para o sistema. O backend executa a inferência utilizando o modelo selecionado e retorna a imagem anotada com as *bounding boxes*, classes detectadas e respectivos níveis de confiança.
- **Modalidade Vídeo:** o usuário fornece um arquivo de vídeo no formato `.mp4`. O sistema processa os quadros (*frames*) sequencialmente, executando a detecção de objetos em cada quadro e gerando como saída um novo vídeo contendo as detecções anotadas, além da taxa média de processamento em FPS (*Frames Per Second*).

- **Modalidade Webcam ao Vivo:** a aplicação acessa localmente a câmera do dispositivo do usuário e realiza inferência em tempo real, alcançando aproximadamente 1 FPS. Diferentemente da execução em ambientes remotos como o Google Colab, o processamento local elimina limitações de acesso contínuo à webcam, proporcionando maior estabilidade e menor latência durante a demonstração.

5 Resultados e Discussão

5.1 Métricas — YOLOv8n

Tabela 5 – YOLOv8n — Base Caneta: média $\pm$ desvio padrão de 5 execuções

Exp.	T_treino (s)	Precision	Recall	F1-Score	mAP@0.50	mAP@0.50:0.95
Exp. 1	91,60 $\pm$ 7,42	0,1527 $\pm$ 0,0544	0,1610 $\pm$ 0,0524	0,1497 $\pm$ 0,0400	0,0623 $\pm$ 0,0177	0,0189 $\pm$ 0,0081
Exp. 2	119,78 $\pm$ 34,41	0,1664 $\pm$ 0,0836	0,1091 $\pm$ 0,0116	0,1240 $\pm$ 0,0146	0,0536 $\pm$ 0,0127	0,0133 $\pm$ 0,0042
Exp. 3	100,17 $\pm$ 19,57	0,2235 $\pm$ 0,0801	0,1558 $\pm$ 0,0331	0,1775 $\pm$ 0,0357	0,0920 $\pm$ 0,0169	0,0266 $\pm$ 0,0088
Exp. 4	100,95 $\pm$ 27,21	0,1747 $\pm$ 0,0794	0,1273 $\pm$ 0,0394	0,1423 $\pm$ 0,0460	0,0628 $\pm$ 0,0288	0,0162 $\pm$ 0,0079
Exp. 5	354,01 $\pm$ 39,52	0,4858 $\pm$ 0,0909	0,3459 $\pm$ 0,0202	0,4009 $\pm$ 0,0271	0,3489 $\pm$ 0,0220	0,1815 $\pm$ 0,0175
Exp. 6	368,46 $\pm$ 12,78	0,4741 $\pm$ 0,0837	0,3503 $\pm$ 0,0255	0,3998 $\pm$ 0,0275	0,3377 $\pm$ 0,0254	0,1765 $\pm$ 0,0209

Fonte: Elaborada pelos autores.

Tabela 6 – YOLOv8n — Base Maça: média $\pm$ desvio padrão de 5 execuções

Exp.	T_treino (s)	Precision	Recall	F1-Score	mAP@0.50	mAP@0.50:0.95
Exp. 1	58,53 $\pm$ 7,61	0,7739 $\pm$ 0,1435	0,6500 $\pm$ 0,0559	0,6991 $\pm$ 0,0734	0,6858 $\pm$ 0,0463	0,4223 $\pm$ 0,0407
Exp. 2	72,23 $\pm$ 24,59	0,8407 $\pm$ 0,0391	0,5717 $\pm$ 0,1020	0,6741 $\pm$ 0,0635	0,6466 $\pm$ 0,0828	0,3815 $\pm$ 0,0681
Exp. 3	62,86 $\pm$ 7,56	0,8132 $\pm$ 0,0547	0,6834 $\pm$ 0,0771	0,7404 $\pm$ 0,0493	0,7003 $\pm$ 0,0599	0,4195 $\pm$ 0,0634
Exp. 4	79,18 $\pm$ 7,48	0,8259 $\pm$ 0,0701	0,6429 $\pm$ 0,1039	0,7145 $\pm$ 0,0409	0,6938 $\pm$ 0,0549	0,3738 $\pm$ 0,0397
Exp. 5	129,72 $\pm$ 5,45	0,8960 $\pm$ 0,0289	0,7695 $\pm$ 0,0514	0,8268 $\pm$ 0,0284	0,7705 $\pm$ 0,0187	0,5408 $\pm$ 0,0349
Exp. 6	127,04 $\pm$ 4,02	0,8805 $\pm$ 0,0562	0,8091 $\pm$ 0,0403	0,8424 $\pm$ 0,0377	0,8077 $\pm$ 0,0268	0,5691 $\pm$ 0,0312

Fonte: Elaborada pelos autores.

5.2 Métricas — SSDLite320 MobileNetV3 Large

Tabela 7 – SSDLite320 — Base Caneta: média $\pm$ desvio padrão de 5 execuções

Exp.	Epochs	T_treino (s)	Precision	Recall	F1-Score	mAP@0.50	mAP@0.50:0.95
Exp. 1	33,6 $\pm$ 1,34	3407,44 $\pm$ 146,86	0,6849 $\pm$ 0,0727	0,2000 $\pm$ 0,0218	0,3090 $\pm$ 0,0299	—	—
Exp. 2	28,4 $\pm$ 0,55	128,20 $\pm$ 5,36	0,5198 $\pm$ 0,0837	0,1922 $\pm$ 0,0425	0,2787 $\pm$ 0,0525	0,2776 $\pm$ 0,0264	0,0874 $\pm$ 0,0074
Exp. 3	34,0 $\pm$ 1,00	156,93 $\pm$ 4,39	0,5156 $\pm$ 0,0464	0,1844 $\pm$ 0,0415	0,2697 $\pm$ 0,0454	0,2520 $\pm$ 0,0230	0,0725 $\pm$ 0,0059
Exp. 4	27,8 $\pm$ 0,45	110,72 $\pm$ 2,28	0,5510 $\pm$ 0,1049	0,1792 $\pm$ 0,0250	0,2676 $\pm$ 0,0273	0,2716 $\pm$ 0,0306	0,0852 $\pm$ 0,0127
Exp. 5	22,0 $\pm$ 0,00	396,63 $\pm$ 1,23	0,6452 $\pm$ 0,1093	0,2052 $\pm$ 0,0577	0,3037 $\pm$ 0,0701	0,3352 $\pm$ 0,0493	0,1184 $\pm$ 0,0206
Exp. 6	22,2 $\pm$ 0,45	403,09 $\pm$ 10,45	0,6994 $\pm$ 0,1098	0,2649 $\pm$ 0,0640	0,3776 $\pm$ 0,0632	0,3824 $\pm$ 0,0427	0,1338 $\pm$ 0,0086

Fonte: Elaborada pelos autores.

Nota: O mAP do Exp. 1 da SSDLite não foi calculado, pois a função de cálculo de mAP foi implementada em versão posterior dos experimentos.

Tabela 8 – SSDLite320 — Base Maçã: média $\pm$ desvio padrão de 5 execuções

Exp.	Epochs	T_treino (s)	Precision	Recall	F1-Score	mAP@0.50	mAP@0.50:0.95
Exp. 1	60,0 $\pm$ 0,00	71,32 $\pm$ 0,17	0,7434 $\pm$ 0,0849	0,8750 $\pm$ 0,0765	0,8016 $\pm$ 0,0709	0,8613 $\pm$ 0,0590	0,4611 $\pm$ 0,0263
Exp. 2	49,2 $\pm$ 6,14	51,32 $\pm$ 6,30	0,7859 $\pm$ 0,0494	0,8125 $\pm$ 0,1768	0,7901 $\pm$ 0,0934	0,8557 $\pm$ 0,0809	0,4719 $\pm$ 0,0664
Exp. 3	60,0 $\pm$ 0,00	78,40 $\pm$ 0,51	0,7272 $\pm$ 0,0685	0,8750 $\pm$ 0,1169	0,7938 $\pm$ 0,0875	0,8787 $\pm$ 0,0508	0,4252 $\pm$ 0,0386
Exp. 4	52,8 $\pm$ 6,42	60,09 $\pm$ 7,82	0,7952 $\pm$ 0,0347	0,8125 $\pm$ 0,0884	0,8008 $\pm$ 0,0391	0,8631 $\pm$ 0,0575	0,4424 $\pm$ 0,1169
Exp. 5	29,2 $\pm$ 1,48	145,67 $\pm$ 7,74	0,8257 $\pm$ 0,0461	0,9875 $\pm$ 0,0280	0,8983 $\pm$ 0,0214	0,9382 $\pm$ 0,0048	0,6256 $\pm$ 0,0158
Exp. 6	28,4 $\pm$ 2,79	142,77 $\pm$ 13,05	0,8401 $\pm$ 0,0474	0,9750 $\pm$ 0,0342	0,9022 $\pm$ 0,0376	0,9499 $\pm$ 0,0182	0,6004 $\pm$ 0,0799

Fonte: Elaborada pelos autores.

5.3 Comparação Final entre as Redes

A Tabela 9 apresenta os melhores resultados obtidos por cada rede em cada base.

Tabela 9 – Melhor resultado de cada rede por base (Exp. 5 ou 6)

Base	Rede	Melhor Exp.	F1-Score	mAP@0.50	mAP@0.50:0.95
Caneta	YOLOv8n	Exp. 5	0,4009	0,3489	0,1815
Caneta	SSDLite320	Exp. 6	0,3776	0,3824	0,1338
Maçã	YOLOv8n	Exp. 6	0,8424	0,8077	0,5691
Maçã	SSDLite320	Exp. 5	0,8983	0,9382	0,6256

Fonte: Elaborada pelos autores.

5.4 Análise dos Resultados

5.4.1 Impacto do Data Augmentation

O *data augmentation* foi o componente que mais impactou positivamente o desempenho em ambas as redes e ambas as bases. A transição do Exp. 1 (linha de base) para o Exp. 5 (*augmentation* + hiperparâmetros modificados) produziu os maiores ganhos observados:

- YOLOv8n Caneta: F1 de 0,15 para 0,40 (+167
- YOLOv8n Maçã: F1 de 0,70 para 0,83 (+19
- SSDLite Caneta: F1 de 0,31 para 0,38 (+22
- SSDLite Maçã: F1 de 0,80 para 0,90 (+12

A adição de rotação ao conjunto de transformações teve papel especialmente relevante, pois canetas frequentemente aparecem em diferentes ângulos nas imagens. A implementação manual da rotação com recálculo correto das *bounding boxes* pelos quatro cantos foi essencial para a validade dos *labels* aumentados.

Do ponto de vista teórico, esse resultado está relacionado à introdução de invariância geométrica durante o treinamento. Redes convolucionais possuem capacidade limitada de lidar naturalmente com rotações significativas dos objetos. Ao incluir rotações artificiais nas imagens de treinamento, o modelo passa a aprender características mais robustas e independentes da orientação espacial.

Além disso, os desvios padrão relativamente baixos observados nas cinco execuções independentes indicam que os ganhos não ocorreram de forma aleatória, mas refletem melhoria consistente da capacidade de generalização das redes.

5.4.2 Impacto do Pré-processamento

O pré-processamento com CLAHE e *blur* gaussiano (Exp. 3 vs. Exp. 1) produziu melhorias modestas, porém consistentes, em ambas as redes:

- YOLOv8n Caneta: F1 de 0,15 para 0,18 (+19)
- YOLOv8n Maçã: F1 de 0,70 para 0,74 (+6)

O efeito foi menos pronunciado que o *augmentation*, indicando que, para bases pequenas, a diversidade de dados é mais determinante que o aprimoramento visual das imagens existentes.

Teoricamente, o filtro gaussiano reduz componentes de alta frequência associadas a ruídos presentes na imagem. Como detectores de características são sensíveis a variações abruptas de intensidade, a suavização reduz ativações espúrias causadas por ruído aleatório.

Entretanto, o excesso de suavização pode degradar bordas importantes para a localização precisa das *bounding boxes*. O ganho moderado observado sugere que o kernel utilizado foi suficientemente pequeno para reduzir ruídos sem comprometer significativamente as estruturas espaciais relevantes dos objetos.

O CLAHE também contribuiu para melhorar o contraste local das imagens, principalmente em regiões com iluminação irregular. A aplicação no canal L do espaço LAB permitiu realçar informações de intensidade sem alterar significativamente as propriedades cromáticas originais das imagens.

5.4.3 Impacto dos Hiperparâmetros Modificados

O Exp. 2 (apenas hiperparâmetros modificados, sem *augmentation* nem pré-processamento) apresentou desempenho inferior ao Exp. 1 em algumas configurações:

- YOLOv8n Caneta: F1 de 0,15 para 0,12 (−19%);
- YOLOv8n Maçã: F1 de 0,70 para 0,67 (−4%).

Isso evidencia que mais épocas e maior *learning rate*, isoladamente, não compensam a falta de diversidade nos dados. A base de canetas, com apenas 350 imagens, foi especialmente afetada.

Os resultados também sugerem possível ocorrência de *overfitting* nas configurações modificadas sem aumento correspondente da variabilidade do conjunto de treinamento. Em bases reduzidas, o aumento excessivo do número de épocas pode levar a rede a memorizar padrões específicos das imagens de treino, reduzindo sua capacidade de generalização em dados não vistos.

5.4.4 Comparação entre Arquiteturas

Nenhuma rede dominou consistentemente as duas bases:

- Na **base de canetas**, a YOLOv8n obteve maior F1-Score (0,40 vs. 0,38), mas a SSDLite320 obteve maior mAP@0.50 (0,38 vs. 0,35);
- Na **base de maçãs**, a SSDLite320 foi claramente superior em F1 (0,90 vs. 0,84) e mAP@0.50 (0,94 vs. 0,81).

O resultado equilibrado entre as arquiteturas é metodologicamente interessante, pois indica que a escolha entre redes de detecção deve considerar a natureza do objeto e do *dataset*, não havendo uma arquitetura universalmente superior para todas as situações.

No caso específico da base de maçãs, o melhor desempenho da SSDLite320 pode estar relacionado ao uso de *transfer learning* com a backbone MobileNetV3 pré-treinada no ImageNet. Como a base contém apenas 97 imagens de treinamento, representações previamente aprendidas tornam-se fundamentais para evitar *overfitting* e melhorar a capacidade de generalização.

Além disso, as maçãs apresentam geometria aproximadamente circular, maior área ocupada na imagem e contraste visual mais evidente em relação ao fundo, características que favorecem maior estabilidade geométrica das *bounding boxes* e maiores valores de IoU.

5.4.5 Dificuldade da Base de Canetas

A caneta se mostrou consistentemente mais difícil de detectar que a maçã, para ambas as redes. O Recall especialmente baixo na base de canetas (0,16–0,27) indica que os modelos perdem uma grande proporção dos objetos reais. Isso se explica por características intrínsecas do objeto: canetas são longas, finas e frequentemente aparecem em cenários complexos com vários outros objetos, papéis com texto e fundos variados. A maçã, por sua vez, é grande, redonda e visualmente destacada.

A dificuldade também pode ser analisada sob a perspectiva da métrica IoU (*Intersection over Union*). Como as canetas ocupam pequena área dentro da *bounding box*, pequenas

imprecisões no posicionamento ou inclinação da caixa delimitadora produzem grandes reduções na área de interseção.

Consequentemente, muitas detecções visualmente corretas acabam apresentando IoU inferior ao limiar de 0,50 utilizado na avaliação, sendo classificadas como falsos positivos. Esse comportamento ajuda a explicar os baixos valores de Recall observados experimentalmente.

5.4.6 Tempo de Inferência

A YOLOv8n foi consistentemente mais rápida que a SSDLite320 em inferência:

- YOLOv8n: 2–4 ms por imagem;
- SSDLite320: 13–66 ms por imagem.

A diferença se deve principalmente ao *pipeline* otimizado da Ultralytics e à menor complexidade da YOLOv8n comparada à MobileNetV3 Large. Ambos os valores são adequados para aplicações em tempo real.

Embora a MobileNetV3 utilize convoluções *depthwise separáveis*, projetadas para reduzir drasticamente o custo computacional em relação às convoluções tradicionais, o desempenho prático depende também do nível de otimização do *framework* utilizado.

A implementação da YOLOv8n pela Ultralytics possui diversas otimizações específicas para inferência em GPU, incluindo operações vetorizadas e melhor aproveitamento do pipeline computacional. Assim, os resultados indicam que eficiência teórica e desempenho real podem diferir dependendo da implementação empregada.

5.4.7 Qualidade da Segmentação Pós-Detecção

A qualidade da segmentação baseada em Otsu, detector de Canny e operações morfológicas mostrou-se diretamente dependente da precisão das *bounding boxes* geradas pelas redes de detecção.

Quando a caixa delimitadora inclui excesso de pixels de fundo, o histograma utilizado pelo método de Otsu passa a ser dominado pelas intensidades do cenário, reduzindo a separabilidade estatística entre objeto e fundo. Como consequência, a limiarização gera máscaras mais ruidosas e menos precisas.

De forma semelhante, bordas irrelevantes presentes no fundo podem ser detectadas pelo operador de Canny, dificultando a obtenção de contornos consistentes do objeto principal.

Esse comportamento evidencia a integração entre técnicas clássicas de Processamento Digital de Imagens e métodos modernos de Aprendizado Profundo: uma etapa de detecção precisa melhora significativamente o desempenho das etapas posteriores de segmentação e análise morfológica.

6 Conclusão

Este projeto apresentou um estudo comparativo entre as arquiteturas YOLOv8n e SSDLite320 MobileNetV3 Large para detecção de objetos, aplicadas às bases de canetas e maçãs, com um *pipeline* experimental de seis configurações e cinco execuções independentes cada.

Os principais achados são:

- O ***data augmentation manual*** foi o componente de maior impacto positivo no desempenho de ambas as redes, especialmente para a base de canetas, que é menor e mais desafiadora. A implementação de rotação com recálculo das *bounding boxes* foi decisiva para os resultados finais;
- O **pré-processamento** (CLAHE + *blur* gaussiano) contribuiu de forma consistente, porém modesta, indicando ser um complemento útil mas não suficiente isoladamente;
- A **separação de *thresholds*** na SSDLite (avaliação vs. operação) foi necessária para alinhar metodologicamente o cálculo de mAP entre as duas redes;
- **Nenhuma rede dominou as duas bases:** a SSDLite foi superior na maçã (F1 0,90 vs. 0,84) e a YOLOv8n ficou levemente à frente na caneta (F1 0,40 vs. 0,38), porém inferior em mAP@0.50, sugerindo que a escolha entre arquiteturas deve considerar as características do objeto e do *dataset*;
- A **base de canetas** foi consistentemente mais difícil para ambas as redes, evidenciando que objetos finos, alongados e em cenários complexos representam um desafio maior para redes de detecção baseadas em *bounding boxes*.

Como trabalhos futuros, sugere-se a implementação de técnicas de *augmentation* mais avançadas (como *mosaic* e *mixup* em implementação manual), o uso de bases de dados maiores para a classe de canetas, e a avaliação de arquiteturas de segmentação de instâncias (como Mask R-CNN) que possam beneficiar objetos com formas complexas.

Referências

- [Gonzalez e Woods 2018]GONZALEZ, R. C.; WOODS, R. E. *Digital Image Processing*. 4. ed. [S.l.]: Pearson, 2018.
- [Howard et al. 2019]HOWARD, A. et al. Searching for MobileNetV3. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. [S.l.: s.n.], 2019. p. 1314–1324.
- [Jocher, Chaurasia e Qiu 2023]JOCHER, G.; CHAURASIA, A.; QIU, J. *Ultralytics YOLOv8*. 2023. Disponível em: <https://github.com/ultralytics/ultralytics>. Acesso em: 2026.
- [Lin et al. 2014]LIN, T.-Y. et al. Microsoft COCO: Common objects in context. In: *European Conference on Computer Vision (ECCV)*. [S.l.: s.n.], 2014. p. 740–755.
- [Liu et al. 2016]LIU, W. et al. SSD: Single shot multibox detector. In: *European Conference on Computer Vision (ECCV)*. [S.l.: s.n.], 2016. p. 21–37.
- [Redmon et al. 2016]REDMON, J. et al. You only look once: Unified, real-time object detection. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2016. p. 779–788.
- [Roboflow]Roboflow. *Apple Detection Dataset*. Disponível em: <https://roboflow.com>. Workspace: marina-vasques-rodrigues, projeto: apple-c3cv9-pnvhy, versão 1. Acesso em: 2026.
- [Roboflow]Roboflow. *Pen Detection Dataset*. Disponível em: <https://roboflow.com>. Workspace: packiot, projeto: pen-wwfeu, versão 1. Acesso em: 2026.

Apêndices

APÊNDICE A – Resultados Visuais das Detecções

Este apêndice apresenta exemplos visuais dos resultados obtidos pelas arquiteturas YOLOv8n e SSDLite320 MobileNetV3 Large durante os experimentos realizados no Google Colab e nos testes da aplicação *web* desenvolvida para demonstração do sistema.

As imagens estão organizadas em resultados gerados durante os experimentos de treinamento/validação e estes práticos realizados na aplicação *web* utilizando envio de imagens.

A.1 Resultados Visuais dos Experimentos

A.1.1 YOLOv8n — Base de Canetas

A Figura 1 apresenta o resultado obtido pela YOLOv8n na base de canetas utilizando a configuração correspondente ao Experimento 7.



Figura 1 – Resultado da YOLOv8n na base de canetas (Exp. 7).

A.1.2 YOLOv8n — Base de Maças

A Figura 3 apresenta o resultado obtido pela YOLOv8n na base de maçãs utilizando a configuração do Experimento 7.

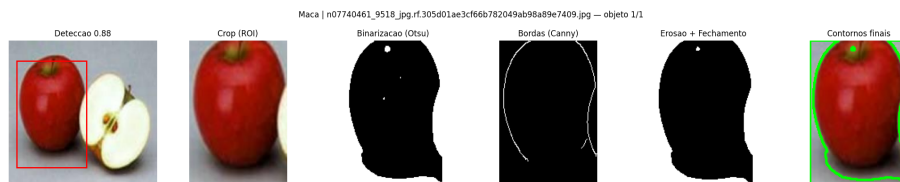


Figura 2 – Exemplo 1 YOLOv8n na base de maçãs (Exp. 7).



Figura 3 – Exemplo 2 YOLOv8n na base de maçãs (Exp. 7).

A.1.3 SSDLite320 — Base de Canetas

A Figura 4 apresenta o resultado obtido pela SSDLite320 MobileNetV3 Large na base de canetas utilizando a configuração do Experimento 7.

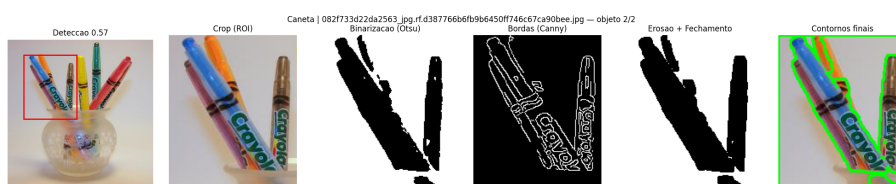


Figura 4 – Resultado da SSDLite320 na base de canetas (Exp. 7).

A.1.4 SSDLite320 — Base de Maçãs

As Figuras 5 e 6 apresentam os resultados obtidos pela SSDLite320 MobileNetV3 Large na base de maçãs utilizando diferentes configurações experimentais.

Maca — Exp. 1 Padrao | n07740461_9518_jpg.rf.305d01ae3cf66b782049ab98a89e7409.jpg | 14.6ms

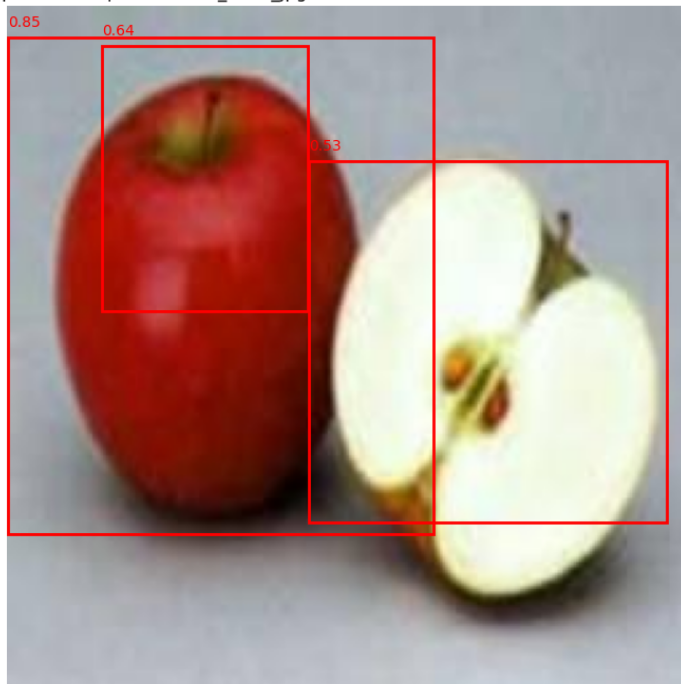


Figura 5 – Resultado da SSDLite320 na base de maçãs (Exp. 1).

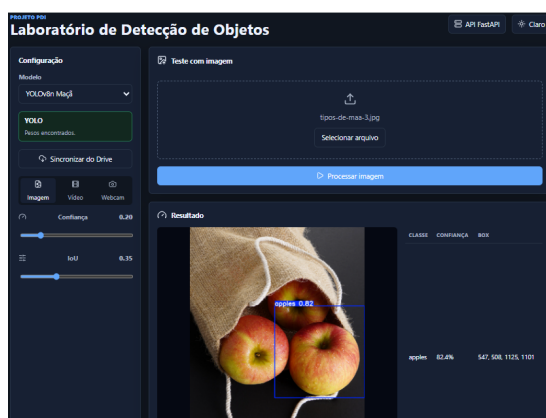


Figura 6 – Resultado da SSDLite320 na base de maçãs (Exp. 7).

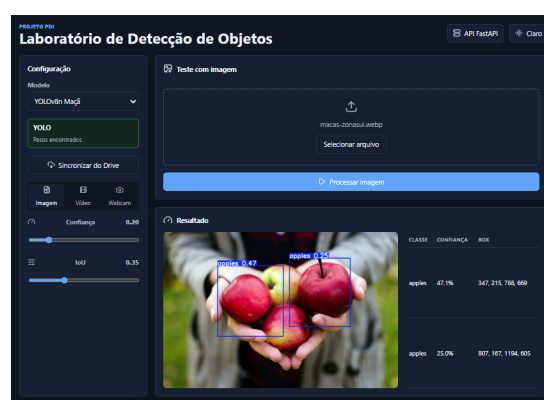
A.2 Testes da Aplicação Web

Esta seção apresenta exemplos de inferência realizados diretamente na aplicação *web*, utilizando imagens enviadas pelo usuário na modalidade de teste por upload.

A.2.1 YOLOv8n — Testes na Base de Maçãs



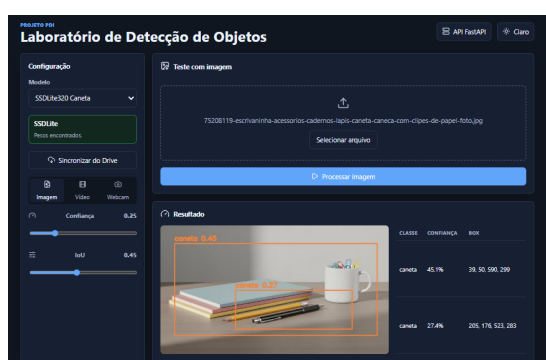
(a) Teste 1



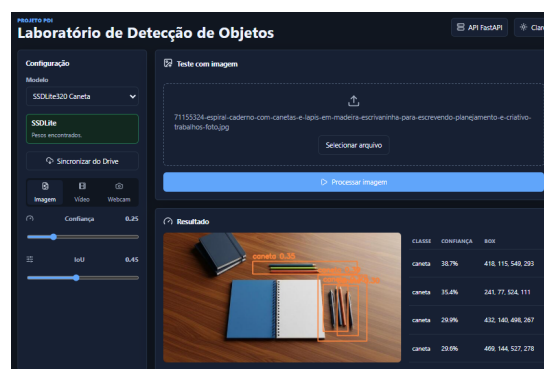
(b) Teste 2

Figura 7 – Inferências realizadas pela aplicação web utilizando YOLOv8n na base de maçãs.

A.2.2 SSDLite320 — Testes na Base de Canetas



(a) Teste 1



(b) Teste 2

Figura 8 – Inferências realizadas pela aplicação web utilizando SSDLite320 na base de canetas.

APÊNDICE B – Estrutura dos Notebooks Colab

Os dois notebooks Colab foram desenvolvidos com estrutura espelhada, contendo as mesmas seções numeradas, conforme o Quadro 10.

Tabela 10 – Estrutura dos notebooks Colab

Seção	Conteúdo
0	<i>Setup</i> : instalação de dependências, <i>imports</i> , <i>seeds</i> , hiperparâmetros
1	<i>Download</i> das bases (Roboflow)
2	Funções auxiliares: <i>dataset</i> , <i>dataloaders</i> , modelo, treino, avaliação
3	Pré-processamento + <i>augmentation</i> + preparação das bases
4	Exp. 1 — Base Caneta (linha de base)
5	Exp. 1 — Base Maçã
6	Exp. 2 — Hiperparâmetros modificados
7	Exp. 3 — Pré-processamento + padrão
8	Exp. 4 — Pré-processamento + modificado
9	Exp. 5 — <i>Data augmentation</i> + modificado
10	Exp. 6 — Tudo combinado (configuração completa)
11	Exp. 7 — Pós-processamento e segmentação
12	Avaliação: imagem, vídeo e <i>webcam</i>
13	Tabela final consolidada

Fonte: Elaborada pelos autores.

APÊNDICE C – Repositório e Organização dos Arquivos

Notebooks:

- `Projeto_PDI_YOLOv8n_DuplaA_FINAL.ipynb`
- `Projeto_PDI_SSDLite320_DuplaB_FINAL.ipynb`

Salvamento automático no Drive:

- Métricas: salvas em pasta `rodada_DATA_HORA` a cada execução (histórico preservado);
- Modelos: os dois melhores modelos de cada rede salvos com nome padronizado para o *front-end*:

- `ssdlite_caneta_best.pth`
- `ssdlite_maca_best.pth`
- `yolo_caneta_best.pt`
- `yolo_maca_best.pt`

Repositório: <https://github.com/guilhermebsantiago/pdi-deteccao-objetos>